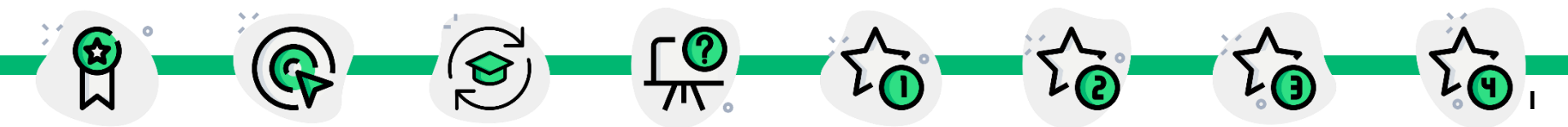


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.06.01

13주차



리스트

◆ 목차

08-1 선형 리스트

08-2 포인터를 이용한 연결 리스트

08-3 커서를 이용한 연결 리스트

08-4 원형 이중 연결 리스트



08-1 선형 리스트

◆선형 리스트 정의하기 - (1)

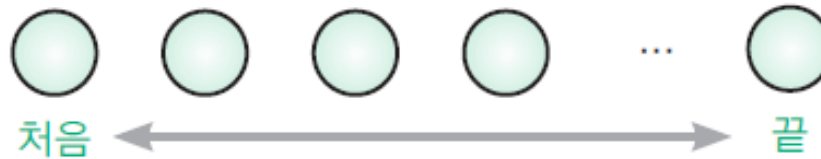
◆리스트list

- 데이터를 순서대로 나열해 놓은 자료구조
- 가장 단순한 구조를 가진 리스트를 선형 리스트(linear list) 또는 연결 리스트(linked list)라고 함
- 그림 8-2를 보면 A부터 F까지 6개의 데이터가 순서대로 나열되어 있고 각각의 데이터가 화살표로 연결되어 있음
- 각 데이터의 연결 관계를 '이야기 전달하기 게임'에 비유할 수 있음
- 즉, 한 사람을 건너뛰어 이야기를 전달할 수 없음



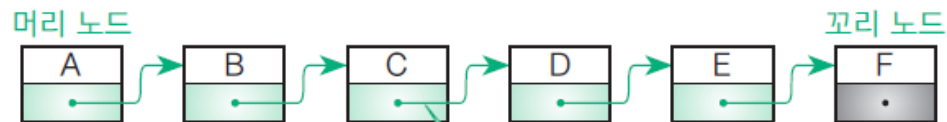
08-1 선형 리스트

리스트는 순서가 있는 데이터를 나열한 구조로 되어 있습니다.



[그림 8-1] 리스트

데이터를 사슬 모양으로 연결한 형태로 되어 있습니다.



다음 노드를 가리키는 포인터를 의미합니다.

[그림 8-2] 연결 리스트



08-1 선형 리스트

◆선형 리스트 정의하기 - (2)

◆노드node

- 리스트의 각 요소(element)
- 각각의 노드는 데이터와 다음 노드를 가리키는 포인터를 가지고 있음
- 처음과 끝에 있는 노드는 특별히 각각 머리 노드(head node), 꼬리 노드(tail node)라고 함
- 하나의 노드에 대해 바로 앞에 있는 노드를 앞쪽 노드(predecessor node), 바로 뒤에 있는 노드를 다음 노드(successor node)라고 함



08-1 선형 리스트

◆배열로 선형 리스트 만들기

◆다음 노드 꺼내기

- 배열의 각 요소에는 연락할 순서대로 데이터가 저장
- 전화를 걸기 위해 필요한 '다음 노드 꺼내기'는 1만큼 큰 인덱스를 갖는 요소에 접근하면 됨

◆노드의 삽입과 삭제

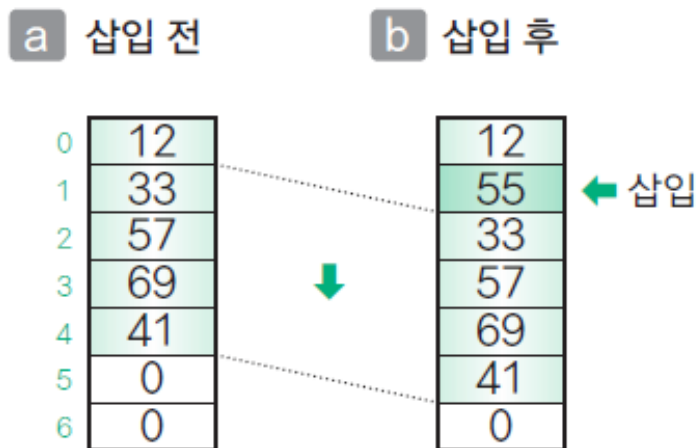
- 회원번호가 55인 회원이 새로 가입했고 이 회원의 정보를 회원번호 12, 33 사이에 삽입하려면 [b]와 같이 삽입 요소 다음의 모든 요소를 하나씩 뒤로 밀어야 함
- 삭제하는 경우도 모든 요소를 뒤로 밀거나 앞으로 당겨야 함
- 이런 작업이 필요하기 때문에 배열로 구현한 선형 리스트는 쌓이는 데이터의 크기를 미리 알아야 함
- 데이터의 삽입, 삭제에 따라 데이터를 모두 옮겨야 하기 때문에 효율이 좋지 않음



08-1 선형 리스트

```
typedef struct { // 회원
    int mem_no;   // 회원번호
    char *name;   // 이름
    char *phone;  // 전화번호
    /* ... */
} Person;

Person data[] = {
    {12, "John", "999-999-1234"},
    {33, "Paul", "999-999-1235"},
    {57, "Mike", "999-999-1236"},
    {69, "Rita", "999-999-1237"},
    {41, "Alan", "999-999-1238"},
    { 0, "", ""},
    { 0, "", ""},
};
```



삽입한 위치 이후의 모든 요소를
하나씩 뒤로 옮깁니다.

[그림 8-3] 배열에 의한 선형 리스트에 삽입



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (1)

◆ 노드용 구조체 Node

- 노드용 객체를, 연결 리스트에 데이터를 삽입할 때 만들고 삭제할 때 없애면, 앞에서 제시한 데이터를 밀고 당기는 문제를 해결할 수 있음
- 노드용 구조체 Node는 다음의 두 멤버, data와 next로 구성
 - data ... 데이터(Member형)
 - next ... 다음 노드에 대한 포인터(자기 자신과 같은 구조체형을 가리키는 포인터형)
- 이와 같이 자기 자신과 같은 자료형의 객체를 가리키는 데이터가 내부에 포함
- 이러한 자료구조를 자기 참조(self-referential)형이라고 함



08-2 포인터를 이용한 연결 리스트

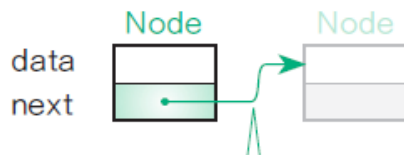
• 완성 파일 chap08/LinkedList.h

Do it! 실습 8-1[A]

```
01 // 포인터로 만든 연결 리스트(헤더)
02 #ifndef __LinkedList
03 #define __LinkedList
04
05 #include "Member.h"
06
07 /*--- 노드 ---*/
08 typedef struct __node {
09     Member      data;    // 데이터
10     struct __node *next; // 뒤쪽 포인터(다음 노드에 대한 포인터)
11 } Node;
12
13 /*--- 연결 리스트 ---*/
14 typedef struct {
15     Node *head;          // 머리 노드에 대한 포인터
16     Node *crnt;          // 선택한 노드에 대한 포인터
17 } List;
18
```

실습 10-1에서 작성

(실습 8-1[B]에서 계속)



자신과 같은 자료형의 객체를 가리킵니다.

[그림 8-4] 연결 리스트를 구현하기 위한 노드의 구조



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (2)

◆ 실습 8-1[B]

Do it! 실습 8-1[B]

• 완성 파일 chap08/LinkedList.h

```
01  /*--- 연결 리스트를 초기화 ---*/
02  void Initialize(List *list);
03
04  /*--- 함수 compare로 x와 같은 노드를 검색 ---*/
05  Node *Search(List *list, const Member *x,
06               int compare(const Member *x, const Member *y));
07
08  /*--- 머리에 노드를 삽입 ---*/
09  void InsertFront(List *list, const Member *x);
10
11  /*--- 꼬리에 노드를 삽입 ---*/
12  void InsertRear(List *list, const Member *x);
13
14  /*--- 머리 노드를 삭제 ---*/
15  void RemoveFront(List *list);
16
17  /*--- 꼬리 노드를 삭제 ---*/
18  void RemoveRear(List *list);
```

```
19
20  /*--- 선택한 노드를 삭제 ---*/
21  void RemoveCurrent(List *list);
22
23  /*--- 모든 노드를 삭제 ---*/
24  void Clear(List *list);
25
26  /*--- 선택한 노드의 데이터를 출력 ---*/
27  void PrintCurrent(const List *list);
28
29  /*--- 선택한 노드의 데이터를 출력(줄 바꿈 문자 포함) ---*/
30  void PrintLnCurrent(const List *list);
31
32  /*--- 모든 노드의 데이터를 리스트 순서대로 출력 ---*/
33  void Print(const List *list);
34
35  /*--- 연결 리스트 종료 ---*/
36  void Terminate(List *list);
37
38  #endif
```

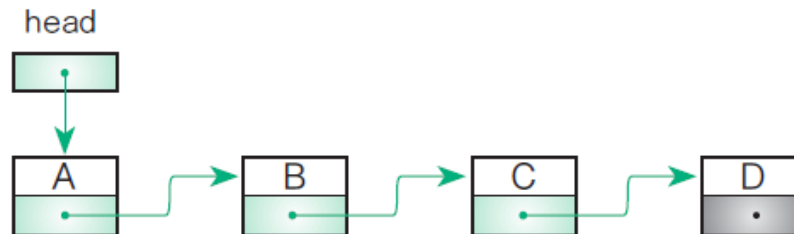


08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (3)

◆ 연결 리스트를 관리하는 구조체 List

- 노드용 구조체 Node를 사용한 연결 리스트를 나타낸 것
- List는 두 멤버로 구성되어 있고 모두 Node에 대한 포인터 자료형을 가지고 있음
- head는 연결 리스트의 머리 노드를 가리키는 머리 포인터이고, crnt는 현재 선택한 노드를 가리키는 선택 포인터
- '검색'한 노드를 선택하고 '삭제'하는 용도로 사용
- 연결 리스트를 관리하는 구조체 List는 2개의 포인터만 가짐
- 그림 8-5에 나타낸 A~D의 각 노드는 Node형 객체이지 List의 일부는 아님



[그림 8-5] 연결 리스트의 이미지



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (4)

◆ 노드를 만드는 AllocNode 함수

- Node형 객체를 만들고 만든 객체의 포인터를 반환

◆ 노드의 멤버값을 설정하는 SetNode 함수

- Node형 객체의 두 멤버(data, next)의 값을 설정하는 함수
- 첫 번째 매개변수 n으로 전달받은 포인터가 가리키는 Node형 객체에 x가 가리키는 값을 대입하고 n의 next에 세 번째 매개변수로 전달받은 next를 대입

◆ 연결 리스트를 초기화하는 Initialize 함수

- 머리 노드를 가리키는 list->head에 널(NULL) 값을 대입하여 노드가 하나도 없는 텅 빈 연결 리스트를 만들



08-2 포인터를 이용한 연결 리스트

• 완성 파일 chap08/LinkedList.c

Do it! 실습 8-2[A]

```
01  /* 포인터로 만든 연결 리스트(소스) */
02  #include <stdio.h>
03  #include <stdlib.h>
04  #include "Member.h"
05  #include "LinkedList.h"
06
07  /*--- 노드를 동적으로 생성 ---*/
08  static Node *AllocNode(void)
09  {
10      return calloc(1, sizeof(Node));
11  }
12  /*--- n이 가리키는 노드의 각 멤버에 값을 설정 ----*/
13  static void SetNode(Node *n, const Member *x, const Node *next)
14  {
15      n->data = *x;      // 데이터
16      n->next = next;    // 뒤쪽 포인터
17  }
18  /*--- 연결 리스트를 초기화 ---*/
19  void Initialize (List *list)
20  {
21      list->head = NULL; // 머리 노드
22      list->crnt = NULL; // 선택 노드
23  }
24
```

실습 10-1에서 작성

(실습 8-2[B]에서 계속)



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (5)

◆ a 연결 리스트가 비어 있는지 판단하는 방법

- 빈 연결 리스트는 노드가 하나도 없는 상태이기 때문에 head가 가리키는 노드도 없음

◆ b 노드가 1개인 연결 리스트를 판단하는 방법

- 머리 포인터 list->head가 가리키는 노드는 머리 노드 A
- 머리 노드 A는 리스트의 꼬리 노드이기도 하므로 next값은 널

◆ c 노드가 2개인 연결 리스트를 판단하는 방법

- 머리 노드는 A, 두 번째 노드이자 꼬리 노드는 B
- 머리 포인터 list->head가 가리키는 노드 A의 뒤쪽 포인터 next는 노드 B를 가리킴
- 꼬리 노드 B의 뒤쪽 포인터 next는 NULL값을 가짐



08-2 포인터를 이용한 연결 리스트

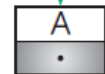
a 노드 0개



NULL

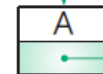
현재 아무런 노드도
가리키고 있지 않습니다.

b 노드 1개

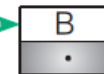


head->next

c 노드 2개



head->next



head->next->next

[그림 8-6] 연결 리스트와 노드 개수에 따른 상태

```
list->head == NULL
```

// 연결 리스트가 비어 있는지(노드가 0개인지) 확인

```
list->head->next == NULL
```

// 노드가 1개인지 확인

```
list->head->next->next == NULL
```

// 노드가 2개인지 확인



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (6)

◆포인터가 머리 노드를 가리키는지 판단하는 방법

- 자료형이 Node *형인 변수 p는 리스트의 노드 중 하나를 가리킴
- 변수 p가 가리키는 노드가 연결 리스트의 머리 노드인지 판단하는 방법은 오른쪽과 같음

◆포인터가 꼬리 노드를 가리키는지 판단하는 방법

- 변수 p가 가리키는 노드가 연결 리스트의 꼬리 노드인지 판단하는 방법은 오른쪽과 같음

```
p == list->head    // p가 가리키는 노드가 머리 노드인지 확인
```

```
p->next == NULL    // p가 가리키는 노드가 꼬리 노드인지 확인
```



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (7)

◆ 검색을 수행하는 Search 함수 - (1)

- 어떤 조건을 만족하는 노드를 검색
- 전달받는 매개변수
 - list ... 검색 대상인 연결 리스트를 가리키는 포인터
 - x ... 검색하는 키값을 저장한 회원 데이터를 가리키는 포인터
 - compare ... 두 번째 매개변수 x가 가리키는 객체와, 연결 리스트의 노드 안의 데이터를 비교하는 함수를 가리키는 포인터로 이 비교 함수는 검색에 성공하면 0을 반환
- 이 함수가 반환하는 값은 찾은 노드에 대한 포인터이고 만약 검색에 실패하면 널을 반환



08-2 포인터를 이용한 연결 리스트

• 완성 파일 chap08/LinkedList.c

Do it! 실습 8-2[B]

```
01  /*--- compare 함수를 사용해 x를 검색 ---*/
02  Node *search(List *list, const Member *x,
03              int compare(const Member *x, const Member *y))
04  {
05      Node *ptr = list->head;
06      while(ptr != NULL) {
07          if(compare(&ptr->data, x) == 0) { // 키값이 같은 경우
08              list->crnt = ptr;
09              return ptr; // 검색 성공
10          }
11          ptr = ptr->next; // 다음 노드를 선택
12      }
13      return NULL; // 검색 실패
14  }
15
```

(실습 8-2[C]에서 계속)

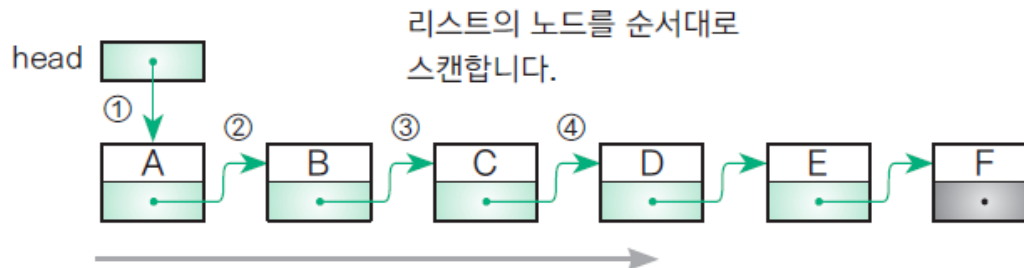


08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (8)

◆검색을 수행하는 Search 함수 - (2)

- 검색에 사용하는 알고리즘은 선형 검색이고, 그림 8-7처럼 검색할 노드를 만날 때까지 머리 노드부터 스캔
- 노드 스캔은 아래의 조건 중 하나만 성립하면 종료
 - 검색 조건을 만족하는 노드를 찾지 못하고 꼬리 노드를 지나가기 직전인 경우
 - 검색 조건을 만족하는 노드를 찾은 경우



[그림 8-7] 연결 리스트에서 노드 D를 선형 검색하는 과정



08-2 포인터를 이용한 연결 리스트

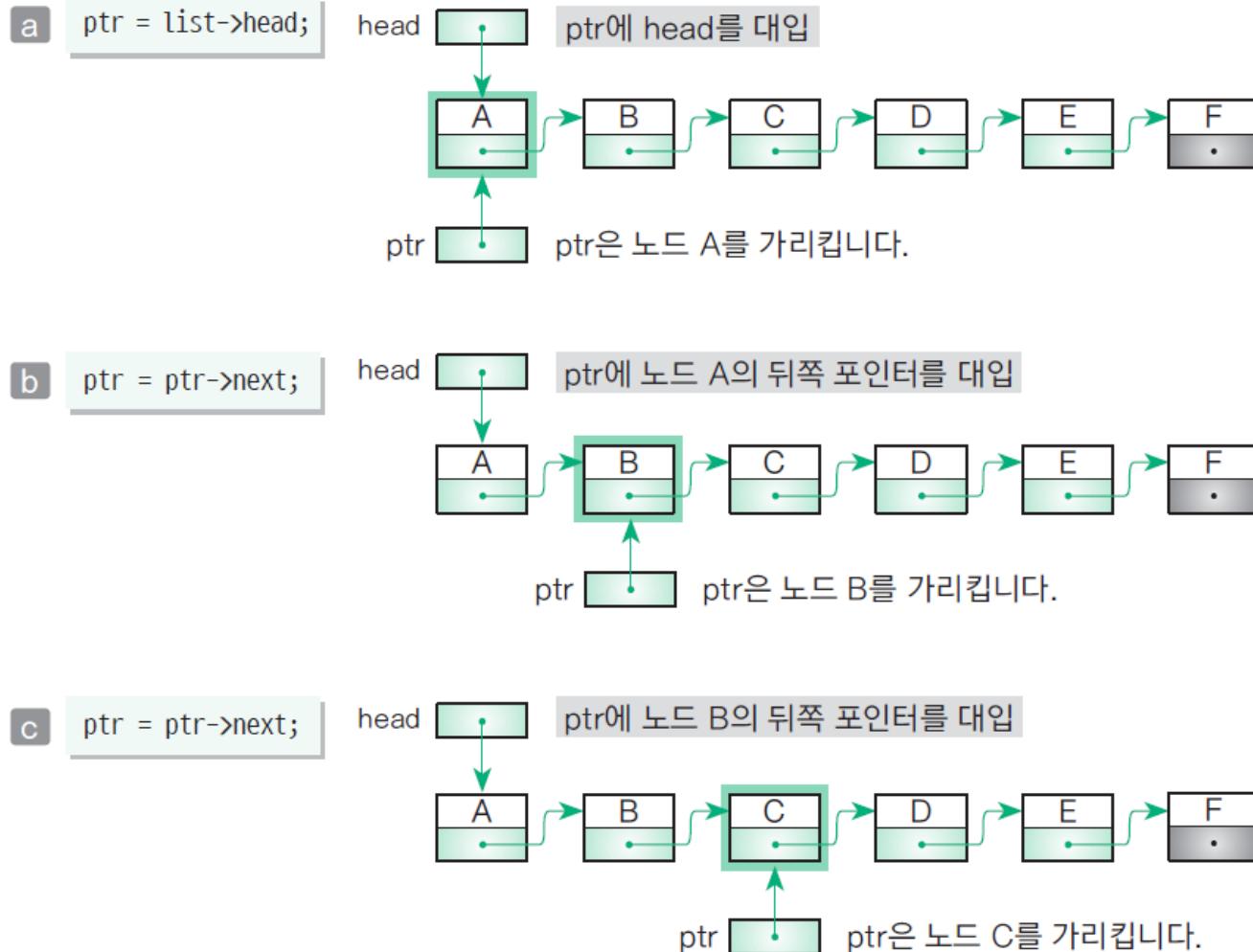
◆ 포인터로 연결 리스트 만들기 - (9)

◆ 검색 과정 - (1)

- 1 스캔하고 있는 노드를 가리키는 포인터 ptr을 list->head로 초기화
 - 그림 a와 같이 ptr이 가리키는 노드는 list->head가 가리키고 있는 머리 노드인 A
- 2 종료 조건 1을 먼저 판단
 - ptr값이 널이 아니면 루프 본문의 3 4를 실행
 - ptr값이 널이면 스캔할 노드가 없음을 의미하기 때문에 while 문을 빠져나와 5로 진행



08-2 포인터를 이용한 연결 리스트



[그림 8-8] 노드 검색



08-2 포인터를 이용한 연결 리스트

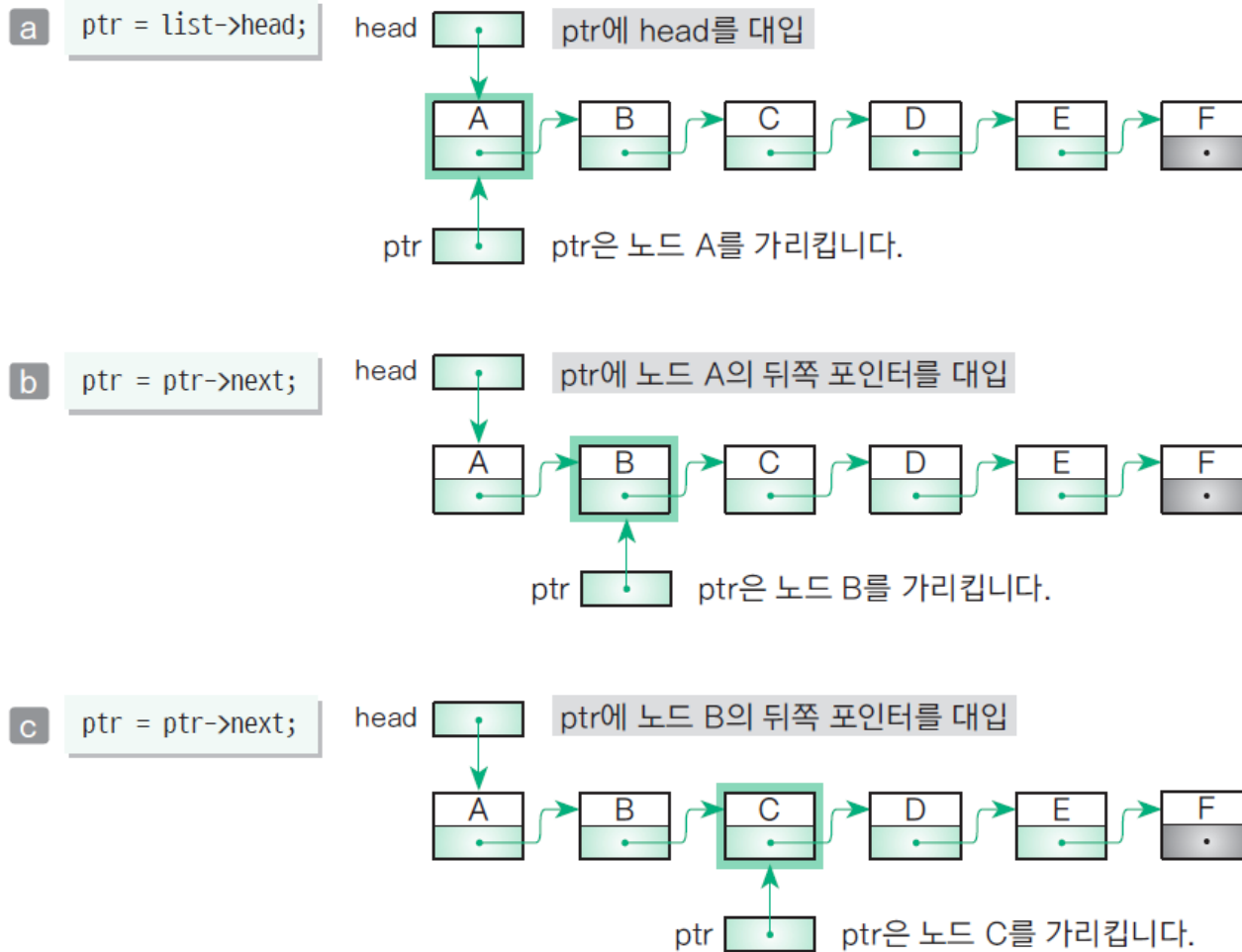
◆ 포인터로 연결 리스트 만들기 - (10)

◆ 검색 과정 - (2)

- 3 종료 조건 2를 판단하기 위해 스캔하고 있는 노드의 데이터(ptr->data)와 x가 가리키는 데이터를 compare 함수로 비교
 - compare 함수는 검색에 성공하면 0을 반환
 - 즉, 종료 조건 2가 성립
 - 곧바로 포인터 list->crnt에 ptr을 대입하고 찾은 노드에 대한 포인터인 ptr을 반환
- 4 ptr에 ptr->next를 대입
 - 이렇게 하면 ptr이 다음 노드를 가리키기 때문에 계속해서 스캔할 수 있음
- 5 검색에 실패하면 널을 반환



08-2 포인터를 이용한 연결 리스트



[그림 8-8] 노드 검색



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (11)

◆머리에 노드를 삽입하는 InsertFront 함수

- 1 삽입 전의 머리 노드 A에 대한 머리 포인터를 ptr에 대입
- 2 삽입할 노드 G를 AllocNode 함수로 만들고 만든 노드 G를 가리키도록 머리 포인터 list->head를 업데이트
- 3 SetNode 함수를 호출하여 값을 설정
 - 이때 삽입한 다음 머리 노드의 다음을 가리키는 포인터의 값을 ptr(삽입하기 전의 머리 노드 A)로 업데이트



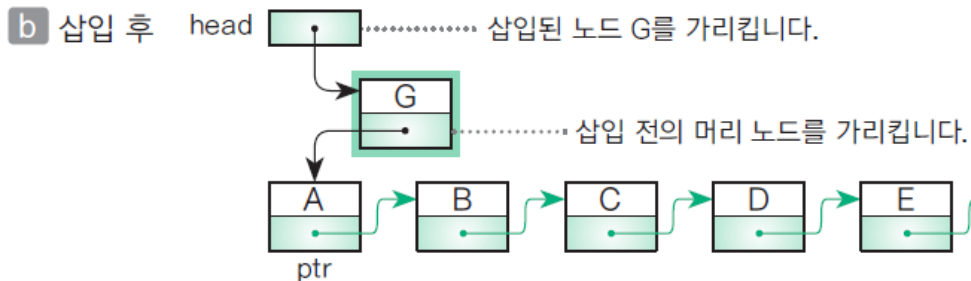
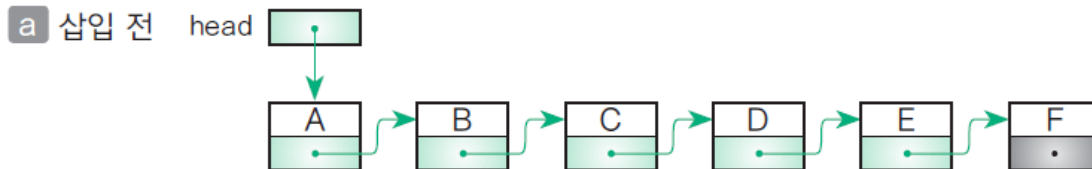
08-2 포인터를 이용한 연결 리스트

Do it! 실습 8-2[C]

• 완성 파일 chap08/LinkedList.c

```
01  *--- 머리에 노드를 삽입 ---*/  
02  void InsertFront(List *list, const Member *x)  
03  {  
04      Node *ptr = list->head; 1  
05      list->head = list->crnt = AllocNode(); 2  
06      SetNode(list->head, x, ptr); 3  
07  }  
08
```

(실습 8-2[D]에서 계속)



[그림 8-9] 머리에 노드를 삽입하는 과정



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (12)

◆꼬리에 노드를 삽입하는 InsertRear 함수 - (1)

- 리스트가 비어 있는지 아닌지 먼저 확인(`list->head == NULL`)하고 경우에 따라 다음의 작업을 수행
 - 1. 리스트가 비어 있는 경우: 머리에 노드를 삽입하는 처리와 같으므로 `InsertFront` 함수로 처리
 - 2. 리스트가 비어 있지 않은 경우: 리스트 꼬리에 노드 G를 삽입(그림 8-10)



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (13)

◆ 꼬리에 노드를 삽입하는 InsertRear 함수 - (2)

- **4** 꼬리 노드를 찾음
 - 머리 노드를 가리키도록 초기화한 ptr이 가리키는 노드를, 계속해서 다음 노드를 가리키도록 업데이트하는 과정을 반복
 - 이렇게 반복하면 노드를 처음부터 차례로 스캔할 수 있음
 - ptr->next가 가리키는 노드가 널이 되면 while 문을 종료
 - 이때 ptr이 가리키는 노드는 꼬리 노드 F.
- **5** 삽입할 노드 G를 AllocNode 함수로 만들
 - 그리고 삽입하기 전의 꼬리 노드 F의 다음 포인터 ptr->next가 가리키는 노드에, 삽입한 다음의 꼬리 노드 G를 대입
 - 그런 다음 SetNode 함수를 호출해 앞에서 만든 노드 G의 값을 설정
 - 이때 노드 G의 다음 노드에 널을 대입



08-2 포인터를 이용한 연결 리스트

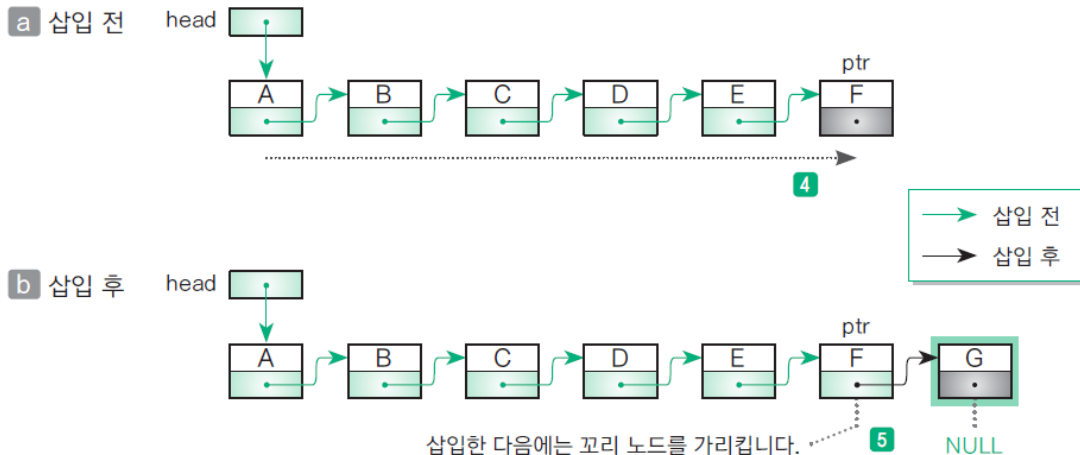
• 완성 파일 chap08/LinkedList.c

Do it! 실습 8-2[D]

```
01  /*--- 꼬리에 노드를 삽입하는 함수 ---*/
02  void InsertRear (List *list, const Member *x)
03  {
04      if(list->head == NULL)      // 비어 있는 경우
05          InsertFront(list, x);    // 머리에 삽입
06      else {
07          Node *ptr = list->head;
08          4 while(ptr->next != NULL)
09              ptr = ptr->next;
10          5 ptr->next = list->crnt = AllocNode();
11              SetNode(ptr->next x, NULL);
12      }
13  }
14
```

while 문이 종료되면 ptr은
꼬리 노드를 가리킵니다.

(실습 8-2[E]에서 계속)



[그림 8-10] 꼬리 노드를 삽입하는 과정



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (14)

◆머리 노드를 삭제하는 RemoveFront 함수

- 리스트가 비어 있지 않은 경우(`list->head != NULL`)에만 삭제를 실행
- 삭제하기 전의 머리 노드 A의 메모리 영역을 해제
- 머리 노드에 대한 포인터 `list->head`에 두 번째 노드 B에 대한 포인터 `list->head->next`를 대입하여 머리 노드에 대한 포인터가 노드 B를 가리키도록 업데이트
- 만약 리스트에 노드가 1개만 있어도 오류 없이 삭제할 수 있고 리스트는 빈 상태가 됨
- 삭제하기 전의 머리 노드는 꼬리 노드이기 때문에 다음 노드를 가리키는 `list->head->next`의 값을 널
- 널을 `list->head`에 대입하면 리스트는 빈 상태가 됨



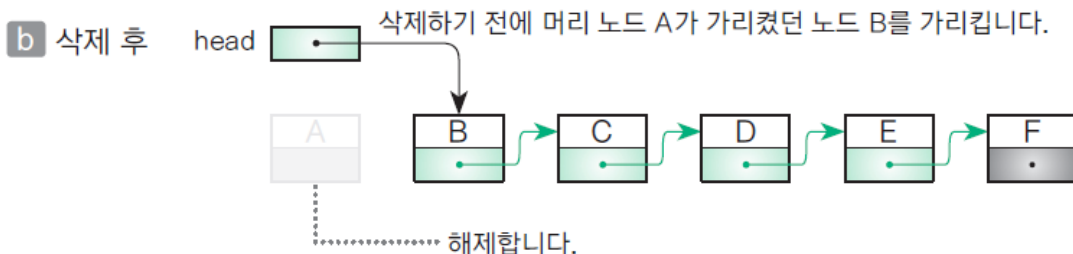
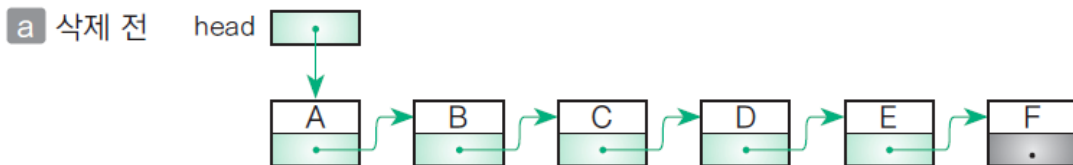
08-2 포인터를 이용한 연결 리스트

• 완성 파일 chap08/LinkedList.c

Do it! 실습 8-2[E]

```
01  /*--- 머리 노드를 삭제 ---*/
02  void RemoveFront(List *list)
03  {
04      if (list->head != NULL) {
05          Node *ptr = list->head->next;    // 2번째 노드에 대한 포인터
06          free(list->head);                // 머리 노드를 해제
07          list->head = list->crnt = ptr;    // 새로운 머리 노드
08      }
09  }
10
```

(실습 8-2[F]에서 계속)



[그림 8-11] 머리 노드를 삭제하는 과정



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (15)

◆ 꼬리 노드를 삭제하는 RemoveRear 함수

- 리스트에 노드가 1개뿐인지(list->next==NULL) 아니면 2개 이상인지에 따라 서로 다른 작업을 수행
 - 1. 리스트에 노드가 1개만 있는 경우: 머리 노드를 삭제하는 것과 같아 RemoveFront 함수로 처리
 - 2. 리스트에 노드가 2개 이상 있는 경우: 그림 8-12의 과정
- **1** '꼬리 노드'와 '꼬리 노드로부터 두 번째 노드'를 찾음
 - 스캔하는 방법은 InsertRear 함수와 비슷하지만 현재 스캔하고 있는 노드의 '앞에 있는 노드'를 가리키는 변수 pre를 추가한 점이 다름
 - while 문이 종료되면 pre는 노드 E를, ptr은 노드 F를 가리킴
- **2** 꼬리 노드부터 두 번째 노드 E의 다음을 가리키는 포인터에 널을 대입하고 꼬리 노드 F의 메모리 영역을 해제



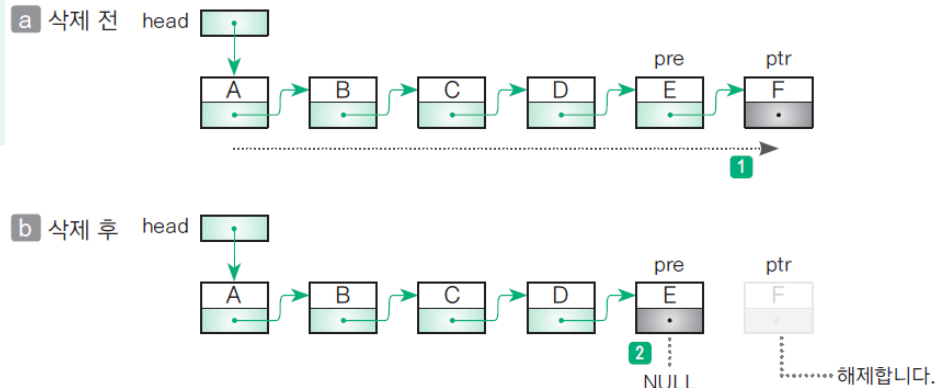
08-2 포인터를 이용한 연결 리스트

Do it! 실습 8-2[F]

• 완성 파일 chap08/LinkedList.c

```
01  /*--- 꼬리 노드를 삭제하는 함수 ---*/
02  void RemoveRear (List *list)
03  {
04      if(list->head != NULL) {
05          if((list->head)->next == NULL)    // 노드가 1개만 있는 경우
06              RemoveFront(list);           // 머리 노드를 삭제
07      }
08      else {
09          Node *ptr = list->head;           1
10          Node *pre;
11          while(ptr->next != NULL) {
12              pre = ptr;
13              ptr = ptr->next;
14          }
15          pre->next = NULL;                 // pre는 꼬리 노드로부터 두 번째 노드
16          free(ptr);                       // ptr은 꼬리 노드           2
17      }
18  }
19 }
20
```

while 문이 종료되면 ptr은 꼬리 노드를 가리키고, pre는 꼬리 노드로부터 두 번째 노드를 가리킵니다.



[그림 8-12] 꼬리 노드를 삭제하는 과정

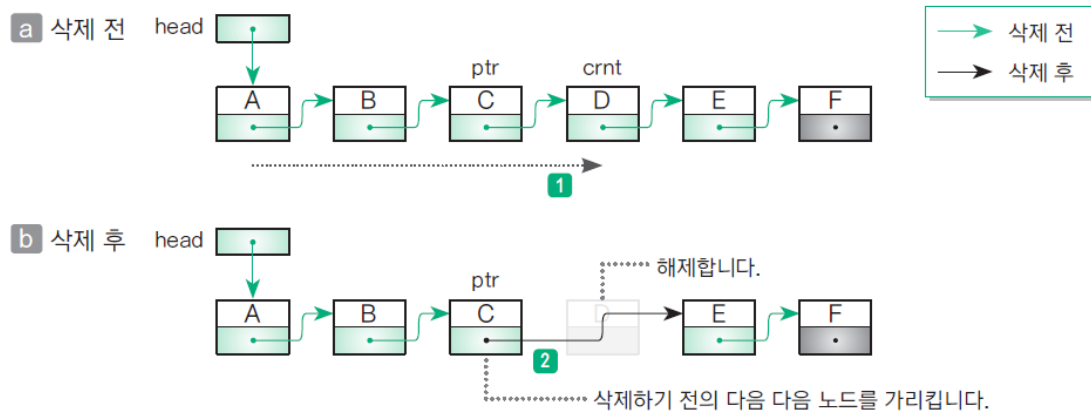


08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (16)

◆선택한 노드를 삭제하는 RemoveCurrent 함수 - (1)

- 현재 선택한 포인터(list->crnt)가 가리키는 노드를 삭제하는 함수
- 삭제할 노드가 머리 노드인지 아닌지에 따라 다음의 작업을 수행
 - 1. crnt가 머리 노드인 경우: 머리 노드를 삭제하면 되므로 RemoveFront 함수로 처리
 - 2. crnt가 머리 노드가 아닌 경우: 그림 8-13의 과정



[그림 8-13] 노드를 삭제하는 과정



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (17)

◆ 선택한 노드를 삭제하는 RemoveCurrent 함수 - (2)

- **1** 선택한 노드의 앞 노드를 찾음
 - while 문은 머리 노드부터 스캔을 시작해 선택한 노드 ptr의 다음 노드를 가리키는 포인터 ptr->next가 list->crnt와 같을 때까지 반복
 - while 문이 종료되고 난 다음 ptr이 가리키는 노드는 삭제하기 위해 선택한 노드 D의 앞쪽 노드인 노드 C가 됨
- **2** 삭제하기 위해 선택한 노드 D의 다음 노드 포인터 list->crnt->next를 노드 C의 다음 노드 포인터 ptr->next에 대입
 - 그러면 노드 C의 다음 노드 포인터가 가리키는 노드가 노드 E로 업데이트됨
 - 그런 다음 노드 D의 메모리 영역을 해제



08-2 포인터를 이용한 연결 리스트

• 완성 파일 chap08/LinkedList.c

Do it! 실습 8-2[G]

```
01  /*--- 선택한 노드를 삭제하는 함수 ---*/
02  void RemoveCurrent (List *list)
03  {
04      if(list->head != NULL) {
05          if(list->crnt == list->head)           // 머리 노드를 선택한 상태라면
06              RemoveFront(list);               // 머리 노드를 삭제
07      } else {
08          Node *ptr = list->head;
09          while(ptr->next != list->crnt)          1
10              ptr = ptr->next;
11          ptr->next = list->crnt->next;          1
12          free(list->crnt);
13          list->crnt = ptr;
14      }
15  }
16  }
17
```

while문이 종료되면 ptr은 선택한
노드의 앞쪽 노드를 가리킵니다.

(실습 8-2[H]에서 계속)



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (18)

◆ 모든 노드를 삭제하는 Clear 함수

- 연결 리스트가 완전히 텅 빈 상태(head == NULL)가 될 때까지 머리 요소의 삭제 작업을 반복

◆ 선택한 노드의 데이터를 출력하는

PrintCurrent / PrintLnCurrent 함수

- 선택한 포인터 list->crnt가 가리키는 노드의 데이터를 출력하는 함수
- 선택한 노드가 없는 경우(list->crnt == NULL)에는 '선택한 노드가 없습니다.'라고 출력

Do it! 실습 8-2[H]

• 완성 파일 chap08/LinkedList.c

```
01  /*--- 모든 노드를 삭제 ---*/
02  void Clear(List *list)
03  {
04      while (list->head != NULL)    // 텅 빌 때까지
05          RemoveFront(list);        // 머리 노드를 삭제
06      list->crnt = NULL;
07  }
08
09  /*--- 선택한 노드의 데이터를 출력 ---*/
10  void PrintCurrent(const List *list)
11  {
12      if (list->crnt == NULL)
13          printf("선택한 노드가 없습니다.");
14      else
15          PrintMember(&list->crnt->data);
16  }
17
18  /*--- 선택한 노드의 데이터를 출력(줄 바꿈 문자 포함) ---*/
19  void PrintLnCurrent(const List *list)
20  {
21      PrintCurrent(list);
22      putchar('\n');
23  }
24
```

(실습 8-2[I]에서 계속)



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (19)

◆ 리스트의 모든 노드를 출력하는 Print 함수

- 머리 노드부터 꼬리 노드까지 포인터 ptr이 가리키는 데이터를 출력

◆ 연결 리스트를 종료하는 Terminate 함수

- 모든 노드를 삭제하는 Clear 함수를 호출

Do it! 실습 8-2[I]

• 완성 파일 chap08/LinkedList.c

```
01  /*--- 모든 노드의 데이터를 리스트 순으로 출력하는 함수 ---*/
02  void Print (const List *list)
03  {
04      if(list->head == NULL)
05          puts("노드가 없습니다.");
06      else {
07          Node *ptr = list->head;
08          puts("【 모두 보기 】");
09          while(ptr != NULL) {
10              PrintLnMember(&ptr->data);
11              ptr = ptr->next;          // 다음 노드를 선택
12          }
13      }
14  }
15  /*--- 연결 리스트를 종료하는 함수 ---*/
16  void Terminate(List *list)
17  {
18      Clear(list);                    // 모든 노드를 삭제
19  }
```



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (20)

◆ 각 함수를 실행한 후의 crnt 값

[표 8-1] 함수를 실행한 후의 선택 노드(crnt) 값

함수	함수를 실행한 후 선택 노드(crnt)가 가리키는 노드
Initialize	NULL
Search	검색에 성공하면 검색한 노드, 검색에 실패하면 업데이트하지 않음
InsertFront	삽입한 머리 노드
InsertRear	삽입한 꼬리 노드
RemoveFront	삭제한 후 머리 노드, 리스트가 비어 있다면 NULL
RemoveRear	삭제한 후 꼬리 노드, 리스트가 비어 있다면 NULL
RemoveCurrent	삭제한 노드의 앞쪽 노드, 리스트가 비어 있다면 NULL
Clear	NULL
PrintCurrent	업데이트하지 않음
PrintLnCurrent	업데이트하지 않음
Print	업데이트하지 않음
Terminate	NULL



08-2 포인터를 이용한 연결 리스트

◆ 포인터로 연결 리스트 만들기 - (21)

◆ 실습 8-3 - (1)

- 연결 리스트를 사용하는 프로그램

Do it! 실습 8-3

• 완성 파일 chap08/LinkedListTest.c

```
01 // 연결 리스트를 사용하는 프로그램
02 #include <stdio.h>
03 #include "Member.h" — 실습 10-1에서 작성
04 #include "LinkedList.h"
05
06 /*--- 메뉴 ---*/
07 typedef enum {
08     TERMINATE, INS_FRONT, INS_REAR, RMV_FRONT, RMV_REAR, PRINT_CRNT,
09     RMV_CRNT, SRCH_NO, SRCH_NAME, PRINT_ALL, CLEAR
10 } Menu;
```

```
12 /*--- 메뉴 선택 ---*/
13 Menu SelectMenu (void)
14 {
15     int ch;
16     char *mstring[] = {
17         "머리에 노드를 삽입",    "꼬리에 노드를 삽입",    "머리 노드를 삭제",
18         "꼬리 노드를 삭제",    "선택한 노드를 출력",    "선택한 노드를 삭제",
19         "번호로 검색",        "이름으로 검색",        "모든 노드를 출력",
20         "모든 노드를 삭제",
21     };
22
23     do {
24         for(int i = TERMINATE; i < CLEAR; i++) {
25             printf("(%2d) %-18.18s ", i + 1, mstring[i]);
26             if((i % 3) == 2)
27                 putchar('\n');
28         }
29         printf("(0) 종료: ");
30         scanf("%d", &ch);
31     } while(ch < TERMINATE || ch > CLEAR);
32     return(Menu)ch;
33 }
```



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (22)

◆실습 8-3 - (2)

```
34  /*--- 메인 ---*/
35  int main (void)
36  {
37      Menu menu;
38      List list;
39      Initialize(&list);          // 연결 리스트를 초기화
40      do {
41          Member x;
42          switch(menu = SelectMenu()) {
43              /* 머리에 노드를 삽입 */
44              case INS_FRONT :
45                  x = ScanMember("머리에 삽입", MEMBER_NO | MEMBER_NAME);
46                  InsertFront(&list, &x);
47                  break;
48              /* 꼬리에 노드를 삽입 */
49              case INS_REAR :
50                  x = ScanMember("꼬리에 삽입", MEMBER_NO | MEMBER_NAME);
51                  InsertRear(&list, &x);
52                  break;
```

```
55          /* 머리 노드를 삭제 */
56          case RMV_FRONT :
57              RemoveFront(&list);
58              break;
59          /* 꼬리 노드를 삭제 */
60          case RMV_REAR :
61              RemoveRear(&list);
62              break;
63          /* 선택한 노드의 데이터를 출력 */
64          case PRINT_CRNT :
65              PrintLnCurrent(&list);
66              break;
67          /* 선택한 노드를 삭제 */
68          case RMV_CRNT :
69              RemoveCurrent(&list);
70              break;
```



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (23)

◆실습 8-3 - (3)

```
75      /* 번호로 검색 */
76      case SRCH_NO :
77          x = ScanMember("검색", MEMBER_NO);
78          if(search(&list, &x, MemberNoCmp) != NULL)
79              PrintLnCurrent(&list);
80          else
81              puts("그 번호의 데이터가 없습니다.");
82
83
84      /* 이름으로 검색 */
85      case SRCH_NAME :
86          x = ScanMember("검색", MEMBER_NAME);
87          if(search(&list, &x, MemberNameCmp) != NULL)
88              PrintLnCurrent(&list);
89          else
90              puts("그 이름의 데이터가 없습니다.");
91          break;
```

```
93      /* 모든 노드의 데이터를 출력 */
94      case PRINT_ALL :
95          Print(&list);
96          break;
97
98      /* 모든 노드를 삭제 */
99      case CLEAR :
100          Clear(&list);
101          break;
102      }
103  } while(menu != TERMINATE);
104  Terminate(&list);          // 연결 리스트 종료
105
106  return 0;
107 }
```



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (24)

◆실습 8-3 실행 결과 - (1)

실행 결과

(1) 머리에 노드를 삽입

(2) 꼬리에 노드를 삽입

(3) 머리 노드를 삭제

(4) 꼬리 노드를 삭제

(5) 선택한 노드를 출력

(6) 선택한 노드를 삭제

(7) 번호로 검색

(8) 이름으로 검색

(9) 모든 노드를 출력

(10) 모든 노드를 삭제

(0) 종료: 1

머리에 삽입하는 데이터를 입력하세요.

번호: 1

이름: 모모

{1, 모모}를 머리에 삽입

(1) 머리에 노드를 삽입

(2) 꼬리에 노드를 삽입

(3) 머리 노드를 삭제

(4) 꼬리 노드를 삭제

(5) 선택한 노드를 출력

(6) 선택한 노드를 삭제

(7) 번호로 검색

(8) 이름으로 검색

(9) 모든 노드를 출력

(10) 모든 노드를 삭제

(0) 종료: 2

꼬리에 삽입하는 데이터를 입력하세요.

번호: 5

이름: 나연

{5, 나연}을 꼬리에 삽입

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
 (4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
 (7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
 (10) 모든 노드를 삭제 (0) 종료: 1

머리에 삽입하는 데이터를 입력하세요.

번호: 10

{10, 정연}을 머리에 삽입

이름: 정연

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
 (4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
 (7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
 (10) 모든 노드를 삭제 (0) 종료: 2

꼬리에 삽입하는 데이터를 입력하세요.

번호: 12

{12, 사나}를 꼬리에 삽입

이름: 사나

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
 (4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
 (7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
 (10) 모든 노드를 삭제 (0) 종료: 1

머리에 삽입하는 데이터를 입력하세요.

번호: 14

{14, 지효}를 머리에 삽입

이름: 지효



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (25)

◆실습 8-3 실행 결과 - (2)

- (1) 머리에 노드를 삽입
- (2) 꼬리에 노드를 삽입
- (3) 머리 노드를 삭제
- (4) 꼬리 노드를 삭제
- (5) 선택한 노드를 출력
- (6) 선택한 노드를 삭제
- (7) 번호로 검색
- (8) 이름으로 검색
- (9) 모든 노드를 출력
- (10) 모든 노드를 삭제
- (0) 종료: 4

꼬리의 {12, 사나}를 삭제

- (1) 머리에 노드를 삽입
- (2) 꼬리에 노드를 삽입
- (3) 머리 노드를 삭제
- (4) 꼬리 노드를 삭제
- (5) 선택한 노드를 출력
- (6) 선택한 노드를 삭제
- (7) 번호로 검색
- (8) 이름으로 검색
- (9) 모든 노드를 출력
- (10) 모든 노드를 삭제
- (0) 종료: 8

검색하는 데이터를 입력하세요.

이름: 사나

그 이름의 데이터가 없습니다.

{사나} 검색 실패

- (1) 머리에 노드를 삽입
- (2) 꼬리에 노드를 삽입
- (3) 머리 노드를 삭제
- (4) 꼬리 노드를 삭제
- (5) 선택한 노드를 출력
- (6) 선택한 노드를 삭제
- (7) 번호로 검색
- (8) 이름으로 검색
- (9) 모든 노드를 출력
- (10) 모든 노드를 삭제
- (0) 종료: 7

검색하는 데이터를 입력하세요.

번호: 10

10 정연

{10} 검색 성공

- (1) 머리에 노드를 삽입
- (2) 꼬리에 노드를 삽입
- (3) 머리 노드를 삭제
- (4) 꼬리 노드를 삭제
- (5) 선택한 노드를 출력
- (6) 선택한 노드를 삭제
- (7) 번호로 검색
- (8) 이름으로 검색
- (9) 모든 노드를 출력
- (10) 모든 노드를 삭제
- (0) 종료: 5

10 정연

선택한 노드는 {10, 정연}

- (1) 머리에 노드를 삽입
- (2) 꼬리에 노드를 삽입
- (3) 머리 노드를 삭제
- (4) 꼬리 노드를 삭제
- (5) 선택한 노드를 출력
- (6) 선택한 노드를 삭제
- (7) 번호로 검색
- (8) 이름으로 검색
- (9) 모든 노드를 출력
- (10) 모든 노드를 삭제
- (0) 종료: 9

【 모두 보기 】

14 지효

10 정연

1 모모

5 나연

모든 노드를 순서대로 출력



08-2 포인터를 이용한 연결 리스트

◆포인터로 연결 리스트 만들기 - (26)

◆실습 8-3 실행 결과 - (3)

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 모든 노드를 삭제 (0) 종료: 7

검색하는 데이터를 입력하세요.

번호: 1

1 모모

{1} 검색 성공

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 모든 노드를 삭제 (0) 종료: 6

선택한 노드 삭제

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 모든 노드를 삭제 (0) 종료: 3

머리 노드 삭제

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 모든 노드를 삭제 (0) 종료: 9

【 모두 보기 】

10 정연

5 나연

모든 노드를 순서대로 출력

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 모든 노드를 삭제 (0) 종료: 0